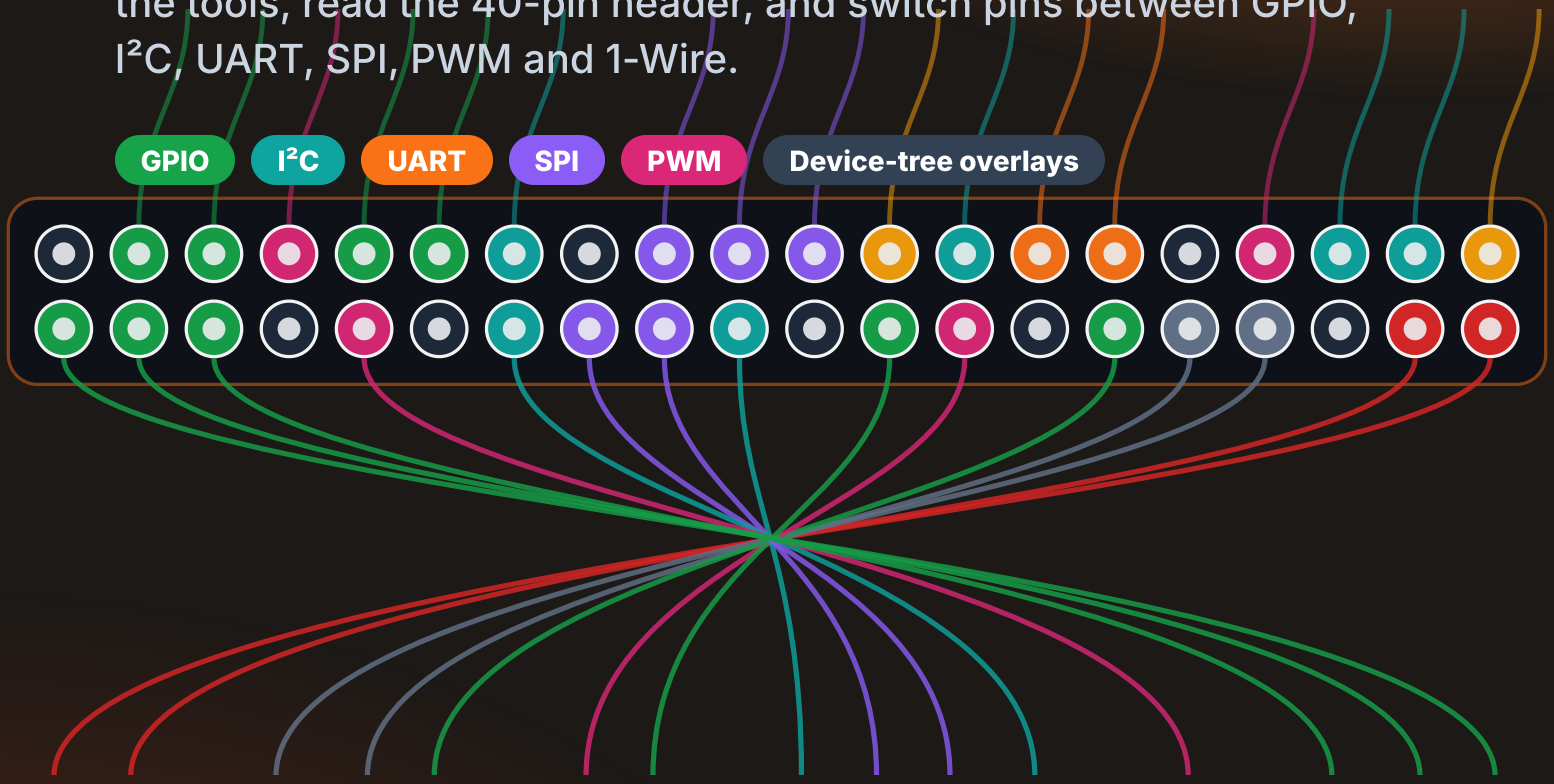
MINTS LAB · HANDS-ON TRAINING

Changing GPIO Pin Functions on the Orange Pi Zero 2W

A step-by-step guide for new undergraduate researchers: install the tools, read the 40-pin header, and switch pins between GPIO, I²C, UART, SPI, PWM and 1-Wire.



PARTS 1–2

Know the board, set it up

Board tour, pinout, what "pin function" means, the four pin-numbering systems, then installing wiringOP, i2c-tools and friends.

PARTS 3–4

Change pins, prove it works

Runtime GPIO control, enabling peripherals with orangepi-config or orangepiEnv.txt, custom overlays, verification tests, troubleshooting.

Based on the Orange Pi Zero 2W User Manual v1.3 and the Orange Pi Linux 6.1 sources

Version 1.0 · October 2026

START HERE

Contents & how to use this guide

Work through it with a board on your desk. Every command block is meant to be typed, and every exercise checks one idea.

PART 0 · BEFORE YOU START

- 0 · Kit list & safety rules 3

PART 1 · KNOW YOUR BOARD

- 1 · Board tour 6
- 2 · The 40-pin header 8
- 3 · What changing a pin's function means 9
- 4 · Four names for every pin 11

PART 2 · SET UP THE SOFTWARE

- 5 · Operating system & first login 13
- 6 · Install the packages 14
- 7 · Reading gpio readall 16

PART 3 · CHANGE PIN FUNCTIONS

- 8 · Two ways to control a pin 18
- 9 · Method A: runtime GPIO with wiringOP 19
- 10 · Method B: orangepi-config 22
- 11 · Method C: edit orangepiEnv.txt 24
- 12 · Prove it works: test each function 26
- 13 · Method D: write your own overlay 28
- 14 · Giving pins back 30

PART 4 · REFERENCE

- 15 · Troubleshooting 33
- 16 · Cheat sheet 34
- A · Full pin-function table 35
- B · Overlay catalogue 36
- Sources 37

By the end you will be able to

- Find any pin on the header and name it four ways (physical, pad, Linux GPIO, wPi).
- Install and use `wiringOP` to read, write and pull up/down any GPIO.
- Explain pin multiplexing and why a pin can be I²C *or* UART, but not both.
- Enable I²C, SPI, UART and PWM with device-tree overlays, and undo them.
- Write and install your own overlay (example: a 1-Wire temperature probe).
- Prove each change worked with a loopback or sensor test.

How to read this guide

Commands sit in dark terminal boxes: type the **bold** text after the prompt; grey lines are the reply. `$` = normal user (add `sudo` for admin commands), `#` = root.

Callouts: **blue** = background, **green** = good habit, **amber** = trap, **red** = can damage hardware. Dashed orange = **exercise**.

Pin colours, used on every diagram: 5 V · 3.3 V · GND · GPIO · I²C · UART · SPI · PWM · debug UART

SECTION 0

Kit list & safety rules

Ten minutes here saves a fried board. Read the safety rules out loud with your lab partner.

What you need

Item	Why
Orange Pi Zero 2W	Any memory size (1, 1.5, 2 or 4 GB) works the same for GPIO.
40-pin male header, soldered	The header is **not soldered at the factory**; ask a lab member to help you solder it.
microSD card, 16 GB or more	Holds the Linux system. Use a good-quality card (Class 10 / A1).
5 V / 2 A (or 3 A) USB-C supply	Plugs into the USB-C power port (see the silkscreen and manual §1.6). Weak phone chargers cause random resets.
3.3 V USB-to-TTL serial adapter (optional)	Lets you see boot messages and log in through the debug UART if networking fails.
Jumper wires (female–female and female–male), breadboard	For the exercises and loopback tests.
LED, 330 Ω resistor, push-button, 4.7 kΩ resistor	Exercise parts. A DS18B20 temperature probe is optional (Section 13).
Multimeter	Checks voltages: the single most useful debugging tool.

Table 1. Kit for this guide.

Safety rules for the 40-pin header

1. **GPIO pins are 3.3 V only.** All 28 GPIO pins use 3.3 V logic. Never connect 5 V to a GPIO pin, even through a sensor's "data" line. Use a level shifter for 5 V devices.
2. **Find pin 1 before wiring.** Pin 1 has a square pad and sits next to the microSD slot. Pins 2 and 4 are 5 V; a wire that lands one row off can put 5 V on a 3.3 V pin.
3. **Power off before you rewire.** Run `sudo poweroff`, wait for the LED to stop, then unplug. Hot-plugging jumper wires can short neighbouring pins.
4. **Never short 5 V or 3.3 V to GND**, and never connect two outputs together.
5. **Keep currents small.** A GPIO pin can light an LED through a resistor. Motors, relays, fans and buzzers need a transistor or driver board.
6. **Touch something grounded first.** Static discharge kills pins silently.

The most common way to destroy an Orange Pi
A jumper wire meant for pin 1 (3.3 V) lands on pin 2 (5 V) and is connected to a GPIO input. Count pins from the square pad every time, and measure with a multimeter before connecting a new sensor.

Powering through the header
The board can also be powered through the 5 V and GND header pins (manual §2.9), but the official recommendation is the USB-C power port. If you do use the header, put 5 V on pin 2 or 4 and GND on pin 6, and double-check polarity.

Pre-power checklist

Tick every box before you plug in the power supply, every time you change the wiring.

- [ ] **Header joints inspected.** No solder bridges between neighbouring pins (use a magnifier).
- [ ] **Every wire checked** against the pinout in Section 2, one wire at a time.
- [ ] **I²C pull-ups go to 3.3 V**, never to 5 V. Many breakout boards already include them.
- [ ] **Power supply is 5 V USB-C**, 2 A or more, plugged into the power port.
- [ ] **Pin 1 found.** Square pad, next to the microSD slot. Count every other pin from here.
- [ ] **Sensor supply matches the sensor.** 3.3 V parts on pin 1 or 17; 5 V (pins 2/4) only to modules whose data lines are 3.3 V-safe.
- [ ] **No loose wire ends** near the header, and no wire touching two pins.
- [ ] **Pins recorded.** Write the pins you used and the overlay you need in your lab notebook.

First measurement after boot
With the multimeter's black probe on pin 6 (GND), pin 1 should read about 3.3 V and pin 2 about 5 V. If either is wrong, power off and find out why before connecting anything else.

PART 1

Know your board

01

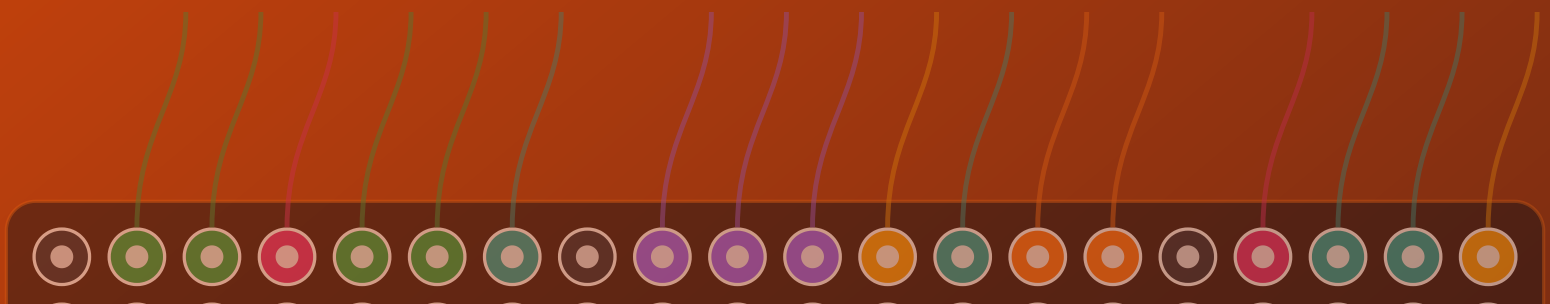
Before changing anything, learn where each pin is, what it can do, and the different numbers people use for the same pin. Most beginner mistakes are numbering mistakes.

1 · Board tour
What is on the board and which side is pin 1

2 · The 40-pin header
Every pin, its power or signal role, and its default function

3 · Pin multiplexing
Why one pin can be GPIO, UART or PWM, and who decides

4 · Four names per pin
Physical number, pad name, Linux GPIO number and wiringPi number

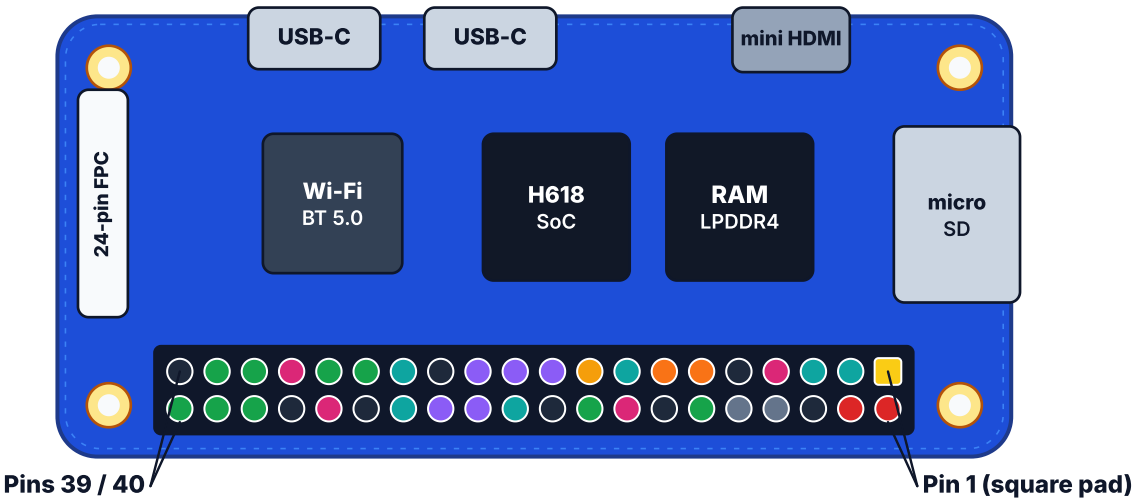


SECTION 1

Board tour

The Orange Pi Zero 2W is a credit-card-thin Linux computer built around the Allwinner H618 chip, the same size as a Raspberry Pi Zero.

Orange Pi Zero 2W, top view (65 × 30 mm), drawn to scale



Odd pins on the inner row, even pins on the board-edge row; pin 1 sits next to the microSD slot.

5 V · 3.3 V · GND · GPIO only · I²C · UART · SPI · PWM · Debug UART0

Figure 1. Top view drawn to scale from the published 65 × 30 mm outline. Pin colours follow the legend; the gold square is pin 1. Component positions are approximate; see manual §1.6 for a photo.

Feature	Orange Pi Zero 2W
Processor (SoC)	Allwinner H618, quad-core 64-bit ARM Cortex-A53, 1.5 GHz
Graphics	Mali-G31 MP2 (OpenGL ES 1.0/2.0/3.2, OpenCL 2.0)
Memory	1, 1.5, 2 or 4 GB LPDDR4 (shared with the GPU)
Storage	microSD slot; 16 MB SPI NOR flash on the bottom side
Wireless	Wi-Fi 5 (802.11a/b/g/n/ac) and Bluetooth 5.0
Ports	Mini HDMI 2.0; two USB-C (one also takes 5 V / 2 A power)
40-pin header	28 GPIO (3.3 V) with UART, I²C, SPI, PWM; plus 5 V, 3.3 V, GND
24-pin FPC connector	For the expansion board: 2 × USB 2.0, 100 Mb Ethernet, IR receiver, audio, power button, 2 keys
Size, weight	65 × 30 × 1.2 mm, 12.5 g

Feature	Orange Pi Zero 2W
Operating systems	Debian 11/12, Ubuntu 20.04/22.04/24.04, Orange Pi OS (Arch), Android 12 TV

Table 2. Hardware summary (Orange Pi Zero 2W user manual v1.3 §1.4 and the official product page).

Where the schematics are
The lab's shared Drive folder has the board schematic (`OPi_ZERO 2W_SCH.pdf`), the expansion-board schematic (`ZERO_2W_24PIN_EXTEND_SCH.pdf`) and a DXF outline. The schematic is the final authority on which chip pad goes to which header pin.

SECTION 2

The 40-pin header

40

Twenty rows of two pins. Twelve are power or ground; the other 28 are general-purpose input/output (GPIO) pins, many of which can also act as a hardware peripheral.

40-pin header pinout (header turned vertical, pin 1 at the top left)

Half-coloured pins can be switched between two peripherals; every coloured pin can also be a plain GPIO.

Function	Name	GPIO	wPi			wPi	GPIO	Name	Function
	3.3V			1	2			5V	
I2C1 SDA	PI8	264	0	3	4			5V	
I2C1 SCL	PI7	263	1	5	6			GND	
PWM3 · UART4 TX	PI13	269	2	7	8	3	224	PH0	UART0 TX (debug)
	GND			9	10	4	225	PH1	UART0 RX (debug)
UART5 TX	PH2	226	5	11	12	6	257	PI1	
UART5 RX	PH3	227	7	13	14			GND	
I2C0 SCL · UART2 TX	PI5	261	8	15	16	9	270	PI14	PWM4 · UART4 RX
	3.3V			17	18	10	228	PH4	
SPI1 MOSI	PH7	231	11	19	20			GND	
SPI1 MISO	PH8	232	12	21	22	13	262	PI6	I2C0 SDA · UART2 RX
SPI1 CLK	PH6	230	14	23	24	15	229	PH5	SPI1 CS0
	GND			25	26	16	233	PH9	SPI1 CS1
I2C2 SDA · UART3 RX	PI10	266	17	27	28	18	265	PI9	I2C2 SCL · UART3 TX
	PI0	256	19	29	30			GND	
	PI15	271	20	31	32	21	267	PI11	PWM1
PWM2	PI12	268	22	33	34			GND	
	PI2	258	23	35	36	24	76	PC12	
	PI16	272	25	37	38	26	260	PI4	
	GND			39	40	27	259	PI3	

5 V · 3.3 V · GND · GPIO only · I²C · UART · SPI · PWM · Debug UART0

Figure 2. Complete pinout. GPIO and wPi numbers match the `gpio readall` output in the user manual (§3.18); alternative functions match the manual's pin table (§3.17) and the Linux 6.1 device tree.

Power pins
5 V: pins 2, 4. 3.3 V: pins 1, 17. GND: pins 6, 9, 14, 20, 25, 30, 34, 39. Power pins are always on; they cannot be reconfigured.

GPIO-only pins
Pins 12, 18, 29, 31, 35, 36, 37, 38, 40 have no peripheral assigned by Orange Pi. They are the safest choice for LEDs, buttons, relays and enable lines.

Debug UART (pins 8, 10)
UART0 is the Linux serial console at 115 200 baud. Do not use these pins for sensors unless you deliberately disable the console (Section 14).

"Name" is a default, not a promise
Labels such as `SDA.1` or `TXD.5` in `gpio readall` describe what the pin *can* be. On a fresh image most signal pins read `OFF` (pad disabled); the debug UART pins 8 and 10 read `ALT2`. A pin only becomes a working I²C or UART pin after you enable that peripheral (Part 3).

SECTION 3

What "changing a pin's function" means

Inside the H618, every pin is connected to a small switch called a multiplexer. Changing a pin's designation simply means choosing a different position on that switch.

The H618 has far more internal hardware (UARTs, I²C controllers, SPI, PWM, Ethernet, audio) than it has pins. So each pin, or **pad**, is shared: a **function-select register** picks which internal block is connected to it. The Linux kernel calls this **pinctrl** (pin control); datasheets call the choices **ALT functions**.

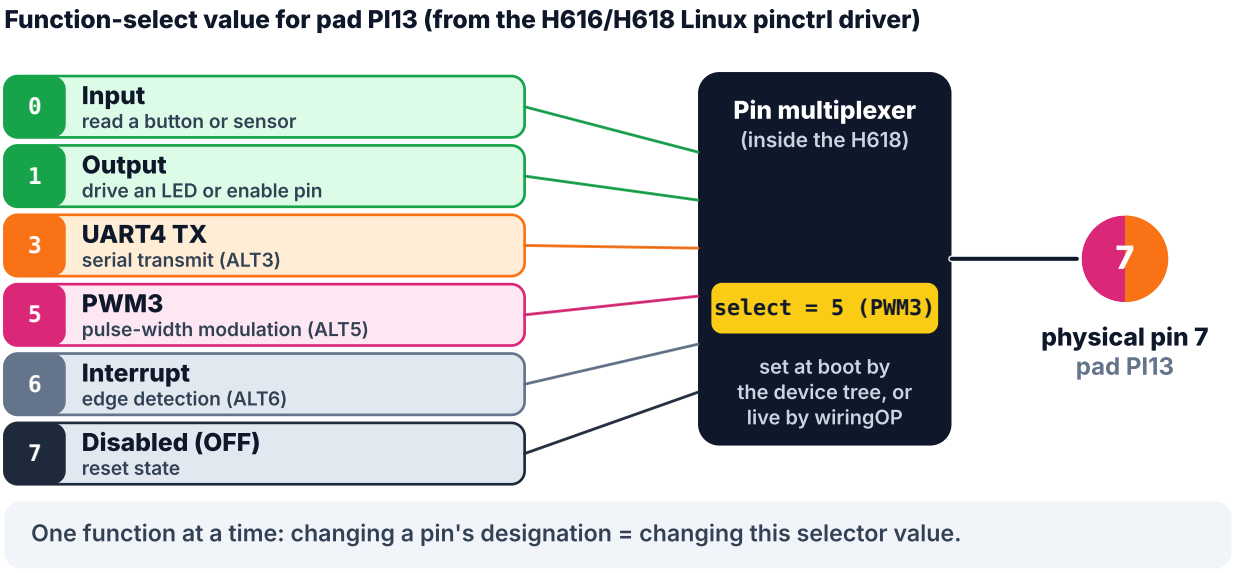


Figure 3. The function selector for pad PI13 (header pin 7). Values come from the Linux pinctrl driver for the H616/H618 family (`pinctrl-sun50i-h616.c`). Selector values 2 and 4 (Ethernet, transport stream) are not used on this board.

Selector values on the H618

0 input · **1** output · **2–5** peripheral functions (ALT2–ALT5) · **6** external interrupt · **7** disabled. This is why `gpio readall` shows modes such as `IN`, `OUT`, `ALT2` and `OFF`.

One function at a time

Pins 15 and 22 (PI5, PI6) can be I²C0 *or* UART2. If you enable both, the second one fails to claim the pins and does not work. Section 10 lists every such pair.

The big picture

There are two separate questions. **(1) Which function is selected on the pad?** wiringOP can change that at runtime. **(2) Is the Linux driver for that peripheral running?** Only the device tree can start a driver, and only a running driver creates a `/dev/i2c-N`, `/dev/spidevX.Y` or `/dev/ttySN` device. To use I²C, SPI or UART from programs you therefore change the device tree (Methods B–D), not just the pad.

SECTION 4

Four names for every pin

Different tools count pins differently. Picking the wrong system is the number-one reason "my pin doesn't respond".

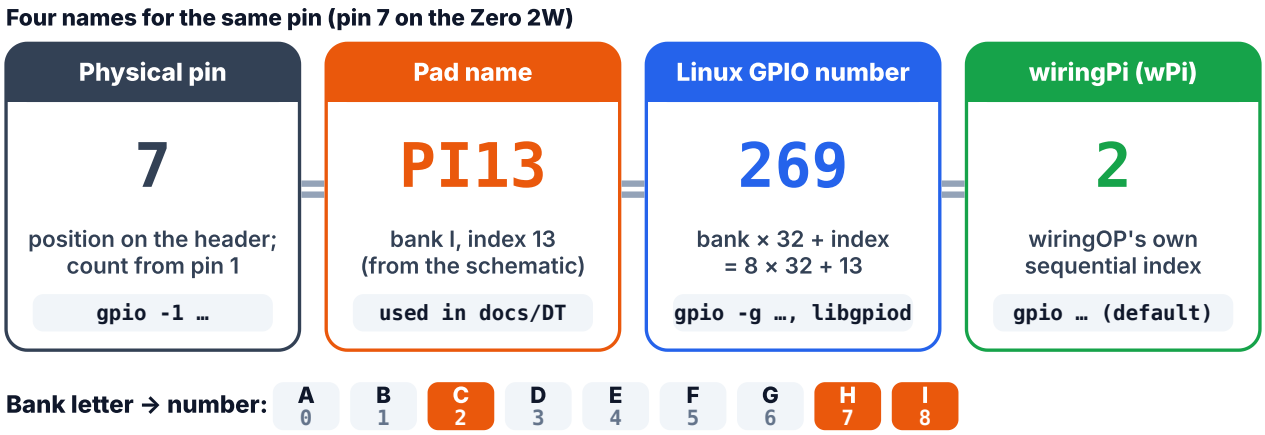


Figure 4. The four numbering systems, using header pin 7 as the example. The Linux GPIO number is computed from the pad name.

System	Who uses it	Example command for pin 7
wiringPi (wPi)	wiringOP `gpio` command and wiringOP-Python (default)	`gpio mode 2 out`
Physical	wiringOP with the `-1` flag; wiring diagrams	`gpio -1 mode 7 out`
Linux GPIO	wiringOP with `-g`; libgpiod tools; kernel messages	`gpio -g mode 269 out`
Pad name	Schematics, device-tree files, the H618 datasheet	`PI13` (in overlays: bank 8, pin 13)

Table 3. Which tool expects which number.

Exercise 4.1: convert

Header pin 22 is pad PI6. (a) What is its Linux GPIO number? (b) Using the pin table, what is its wPi number? (c) Which command sets it as an input in wPi numbering?

Answers: (a) 8 × 32 + 6 = 262 (b) 13 (c) `gpio mode 13 in`

PART 2

Set up the software

02

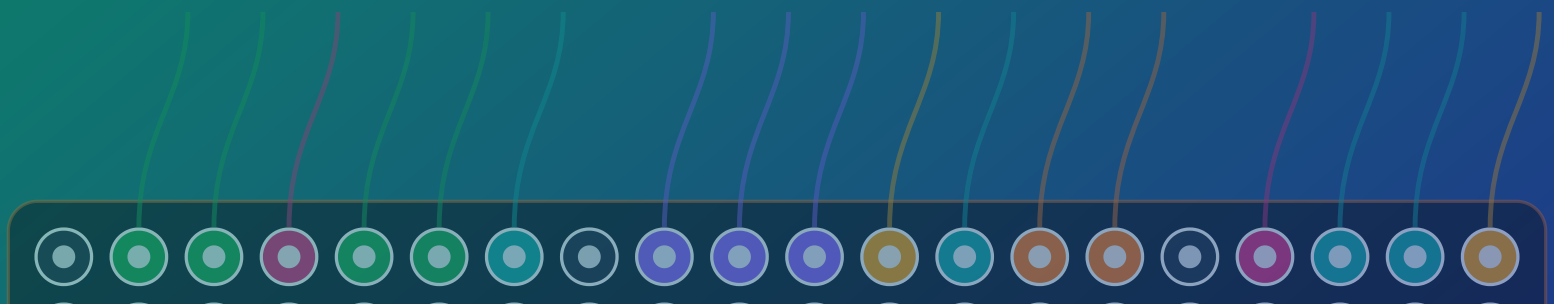
Get an Orange Pi Linux image running, log in, and install the four tools used in the rest of the guide: wiringOP, wiringOP-Python, i2c-tools and the device-tree compiler.

5 · OS & first login
Which image to choose, flashing the card, SSH or serial login

6 · Install the packages
apt packages, checking or building wiringOP, wiringOP-Python

7 · Reading gpio readall
The one command that shows the state of every pin

Time needed
About 45 minutes including flashing the card



SECTION 5

Operating system & first login

6.1

Use an official Orange Pi Debian or Ubuntu image with the Linux 6.1 kernel. All commands in this guide were written for those images.

Image	Kernel	Use it?
Debian 12 Bookworm	Linux 6.1	**Recommended** (server image for sensor nodes)
Ubuntu 22.04 Jammy · Ubuntu 24.04 Noble	Linux 6.1	Good
Debian 11 Bullseye	Linux 6.1	Works, but cannot build kernel modules (manual §3.32)
Ubuntu 20.04 Focal · Debian 11 Bullseye	Linux 5.4	Older; some paths and device names differ (noted where relevant)

Table 4. Linux images offered for the Zero 2W (manual §3.1). The kernel version appears at the end of the image file name, e.g. `…_linux6.1.31.7z`.

1 Download the image

From the official Orange Pi Zero 2W page (orangepi.org → Orange Pi Zero 2W → Downloads), get the Debian or Ubuntu image and unpack the `.7z` file to get an `.img`.

2 Flash the microSD card

Use **balenaEtcher** (Windows, macOS, Linux): select the `.img`, select the card, Flash. It verifies the write automatically (manual §2.3).

3 Boot and connect

Insert the card, plug in power. The first boot expands the file system, so wait a minute. Then either connect a keyboard and mini-HDMI screen, or log in over the network with SSH, or use the debug serial port.

```
# from your laptop, on the same network
you@laptop:~$ ssh orangepi@<board-ip-address>
password: orangepi
orangepi@orangepizero2w:~$ passwd   # change it!
```

Default accounts: `orangepi` / `orangepi` and `root` / `orangepi` (manual §3.4.1). Change both passwords on any board that goes on a network.

No network yet? Use the debug UART

Connect a 3.3 V USB-to-TTL adapter: adapter **GND → pin 6**, adapter **RX → pin 8** (board TX), adapter **TX → pin 10** (board RX). Open PuTTY or MobaXterm at **115200** baud, no flow control (manual §2.8). TX and RX must cross; if you see nothing, swap them.

Update before you start
Run `sudo apt update && sudo apt upgrade -y` once, then reboot. This also updates `orangepi-config` and the kernel package.

SECTION 6

Install the packages

Most tools are already on the official image. This section checks each one and installs whatever is missing.

apt

Tool	Package / source	What it is for
wiringOP (`gpio` command)	pre-installed; source on GitHub	Read/write GPIO, set pull-ups, set PWM, print `gpio readall`. Orange Pi's port of wiringPi.
wiringOP-Python	GitHub (build)	The same functions from Python: `import wiringpi`.
`orangepi-config`	pre-installed	Menu tool that enables or disables device-tree overlays.
i2c-tools	`apt`	`i2cdetect`, `i2cget`, `i2cset`: scan and talk to I²C sensors.
gpiod	`apt`	Standard Linux GPIO tools (`gpiodetect`, `gpioinfo`) to see which lines the kernel has claimed.
device-tree-compiler	`apt`	`dtc`: compiles your own overlay files (Section 13).
git, swig, python3-dev, python3-setuptools	`apt`	Needed to build wiringOP and wiringOP-Python from source.

Table 5. Software used in this guide.

1 Install the apt packages

```
orangepi@orangepizero2w:~$ sudo apt update
orangepi@orangepizero2w:~$ sudo apt install -y git i2c-tools gpiod device-tree-compiler
orangepi@orangepizero2w:~$ sudo apt install -y swig python3-dev python3-setuptools
```

2 Check that wiringOP is there

Official images ship with wiringOP. If this prints a table of 40 pins, skip to step 4.

```
orangepi@orangepizero2w:~$ gpio -v          # prints the wiringOP version
orangepi@orangepizero2w:~$ gpio readall     # should say ZERO2W in the header line
```

3 Build wiringOP from source (only if missing, or for hardware PWM)

The manual (§3.20) notes that the wiringOP shipped with the first image release (v1.0.0) lacks hardware-PWM support, so rebuild it if you need `gpio mode … pwm`. Use the **`next`** branch.

```
orangepi@orangepizero2w:~$ git clone https://github.com/orangepi-xunlong/wiringOP.git
-b next
orangepi@orangepizero2w:~$ cd wiringOP
orangepi@orangepizero2w:~/wiringOP$ sudo ./build clean
orangepi@orangepizero2w:~/wiringOP$ sudo ./build
orangepi@orangepizero2w:~/wiringOP$ gpio readall
```

No internet access from GitHub? The image keeps a copy of the source in `/usr/src/wiringOP`.

4 Install wiringOP-Python

```
orangepi@orangepizero2w:~$ git clone --recursive https://github.com/orangepi-
xunlong/wiringOP-Python -b next
orangepi@orangepizero2w:~$ cd wiringOP-Python
orangepi@orangepizero2w:~/wiringOP-Python$ git submodule update --init --remote
orangepi@orangepizero2w:~/wiringOP-Python$ python3 generate-bindings.py > bindings.i
orangepi@orangepizero2w:~/wiringOP-Python$ sudo python3 setup.py install
orangepi@orangepizero2w:~/wiringOP-Python$ python3 -c "import wiringpi; print('ok')"
```

Offline copy: `/usr/src/wiringOP-Python`. Newer Python versions may print setuptools deprecation warnings; they do not stop the install.

wiringOP needs root

wiringOP drives the hardware registers directly through `/dev/mem`, so `gpio` commands that change pins and any Python script using `wiringpi` generally need `sudo` (or a root shell). The manual runs all wiringOP-Python examples as root.

SECTION 7

Reading gpio readall

Run it before and after every change. It shows the number, current mode and current logic level of every pin in one screen.

Reading gpio readall (sample from the Orange Pi Zero 2W user manual v1.3)

```
orangepi@orangepizero2w:~$ gpio readall
```

GPIO	wPi	Name	Mode	V	Physical		V	Mode	Name	wPi	GPIO
		3.3V			1	2			5V		
264	0	SDA.1	OFF	0	3	4			5V		
263	1	SCL.1	OFF	0	5	6			GND		
269	2	PWM3	OFF	0	7	8	0	ALT2	TXD.0	3	224
		GND			9	10	0	ALT2	RXD.0	4	225
226	5	TXD.5	OFF	0	11	12	0	OFF	PI01	6	257
227	7	RXD.5	OFF	0	13	14			GND		
261	8	TXD.2	OFF	0	15	16	0	OFF	PWM4	9	270
		3.3V			17	18	0	OFF	PH04	10	228
231	11	MOSI.1	OFF	0	19	20			GND		
232	12	MISO.1	OFF	0	21	22	0	OFF	RXD.2	13	262
230	14	SCLK.1	OFF	0	23	24	0	OFF	CE.0	15	229
		GND			25	26	0	ALT3	CE.1	16	233
266	17	SDA.2	OFF	0	27	28	0	OFF	SCL.2	18	265
256	19	PI00	OFF	0	29	30			GND		
271	20	PI15	OFF	0	31	32	0	OFF	PWM1	21	267
268	22	PI12	OFF	0	33	34			GND		
258	23	PI02	OFF	0	35	36	1	OUT	PC12	24	76
272	25	PI16	OFF	0	37	38	0	OFF	PI04	26	260
		GND			39	40	0	OFF	PI03	27	259

GPIO
Linux GPIO number (bank × 32 + index)

wPi
wiringPi number used by plain gpio commands

Mode
OFF, IN, OUT, or ALTn = a peripheral owns it

V
logic level read right now (0 or 1)

Physical
header position (1–40)

Name
label = default/most common function

ALT2 on pins 8/10: the debug UART0 owns PH0/PH1. OFF: the pad is disabled until something configures it.
Your board may show different Mode/V values depending on image version and enabled overlays.

Figure 5. Annotated `gpio readall` output, transcribed from the user manual. The left half lists the odd (inner-row) pins, the right half the even pins, exactly as they sit on the board.

Exercise 7.1: read the table

Using the output above: (a) Which wPi number controls header pin 32? (b) What mode are pins 8 and 10 in, and why? (c) What is the Linux GPIO number of the pin labelled `CE.0`?

Answers: (a) 21 (PWM1) (b) ALT2: the debug UART0 uses them (c) 229 (pin 24, PH5)

PART 3

Change pin functions

Four methods, from quickest to most powerful. Use Method A to flip plain GPIO pins. Use B or C to turn pins into I²C, SPI, UART or PWM. Use D when the function you need is not in the list.

A · wiringOP at runtime

Input, output, pull-up/down, PWM. Instant, but forgotten at reboot.

B · orangepi-config

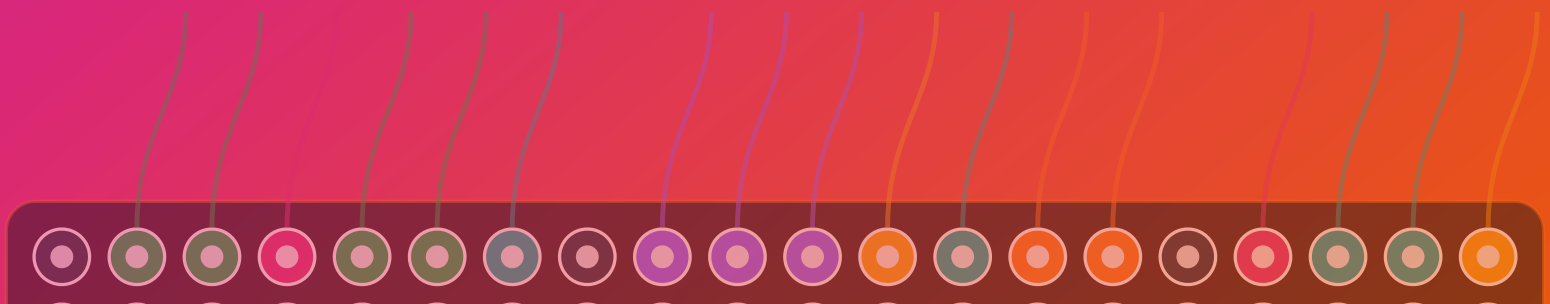
Tick a box to enable an I²C, SPI, UART or PWM overlay. Survives reboot.

C · Edit orangepiEnv.txt

The same as B, by hand. Scriptable and easy to copy between boards.

D · Your own overlay

Anything the device tree can describe: 1-Wire, bus speed, extra chip selects…



SECTION 8

Two ways to control a pin

wiringOP pokes the hardware directly while Linux runs. Device-tree overlays tell Linux at boot which peripherals exist and which pins they own.

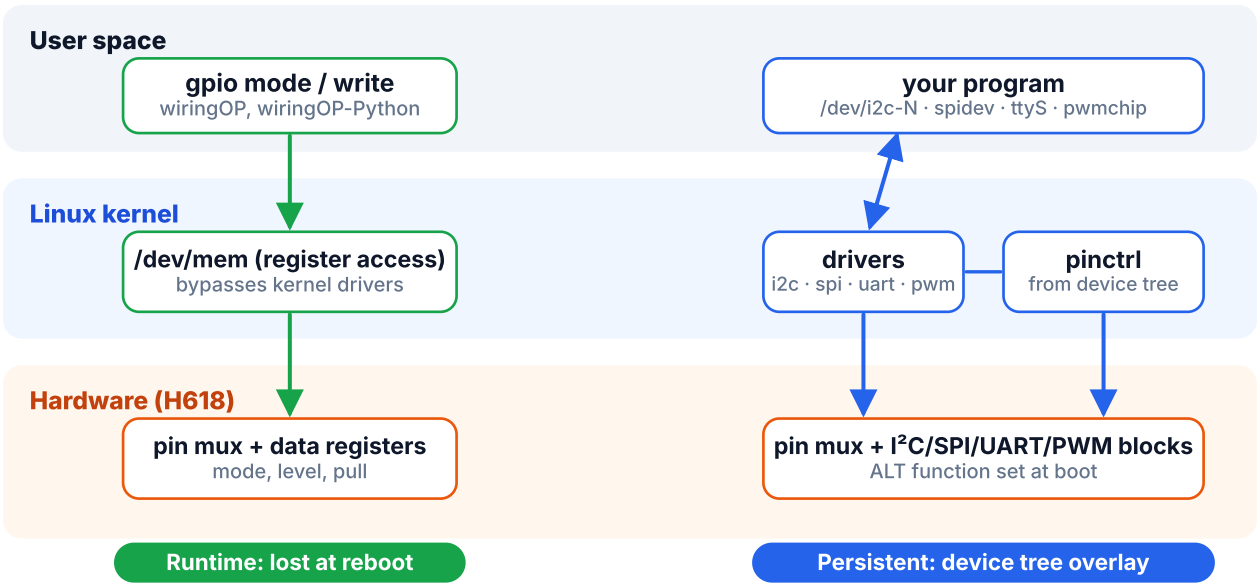


Figure 6. The runtime path (left) changes the pad's mode and level directly. The persistent path (right) selects the pad function at boot and starts the kernel driver that provides the `/dev` device your program opens.

You want to…	Use	Why
Blink an LED, read a button, drive an enable pin	Method A (wiringOP)	No driver needed; plain GPIO.
Talk to an I²C, SPI or UART sensor	Method B or C (overlay)	You need the kernel driver and its `/dev` node.
Make a PWM signal	wiringOP `gpio mode … pwm`, or overlay + sysfs	Both work; the overlay route uses the kernel PWM driver.
Use a 1-Wire temperature probe, change I²C speed	Method D (custom overlay)	Not in the ready-made list.
Keep a GPIO setting after reboot	Method A + a systemd service	wiringOP settings live only until power-off.

Table 6. Choosing a method.

Do not mix the two paths on one pin
wiringOP can switch the mode of a pin that a kernel driver owns (for example pins 8 and 10, the serial console) and the driver will not notice. That silently breaks the driver. Leave pins that belong to an enabled overlay alone.

SECTION 9

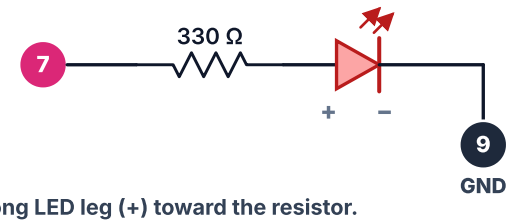
Method A: runtime GPIO with wiringOP

The `gpio` command changes a pin immediately. Examples use wPi numbers, as in the manual.

Command	What it does
`gpio mode <wPi> out`	Make the pin an output (mode becomes `OUT`).
`gpio write <wPi> 1 | 0`	Drive the output high (3.3 V) or low (0 V).
`gpio toggle <wPi>`	Flip the output.
`gpio mode <wPi> in`	Make the pin an input.
`gpio mode <wPi> up | down | tri`	Enable the internal pull-up, pull-down, or no pull (`off` also works).
`gpio read <wPi>`	Print the current level, 0 or 1.
`gpio mode <wPi> pwm`	Hardware PWM on PWM-capable pins (wPi 21, 22, 2, 9 = PWM1–4).
`gpio mode <wPi> alt2 … alt7`	Set the raw function selector (advanced; does not start a driver).
`gpio -1 … / gpio -g …`	Use physical pin numbers / Linux GPIO numbers instead of wPi.

Table 7. wiringOP commands for changing a pin at runtime (manual §3.19–3.20 and the wiringOP source).

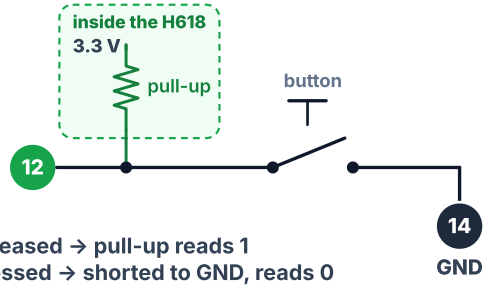
Output: LED on pin 7 (PI13, wPi 2)



Long LED leg (+) toward the resistor.
I ≈ (3.3 – 2.0) V / 330 Ω ≈ 3.9 mA (red LED)

```
gpio mode 2 out
gpio write 2 1   # LED on
gpio write 2 0   # LED off
```

Input: push-button on pin 12 (PI1, wPi 6)



Released → pull-up reads 1
Pressed → shorted to GND, reads 0

```
gpio mode 6 in
gpio mode 6 up    # enable pull-up
gpio read 6       # 1 = released
```

Figure 7. Two starter circuits. The LED uses pin 7 because the manual uses it as its GPIO example; the button uses pin 12, a GPIO-only pin. The internal pull-up means the button needs no external resistor.

Exercise 9.1: LED and multimeter

Build the LED circuit. Run `gpio mode 2 out`, then `gpio write 2 1`: the LED lights and your meter reads about 3.3 V on pin 7. `gpio write 2 0` gives 0 V. Run `gpio readall` and confirm pin 7 now shows `OUT` with V = 0 or 1.

Exercise 9.2: pull-up and pull-down without any wiring

Leave pin 7 unconnected. `gpio mode 2 in`, `gpio mode 2 up`, `gpio read 2` → 1. Then `gpio mode 2 down`, `gpio read 2` → 0. The internal resistor alone sets the level (manual §3.19.2).

The same thing in Python

```
# blink_button.py  (run with: sudo python3 blink_button.py)
import wiringpi, time
from wiringpi import GPIO

LED, BUTTON = 2, 6                # wPi numbers: header pins 7 and 12
wiringpi.wiringPiSetup()          # use wPi numbering
wiringpi.pinMode(LED, GPIO.OUTPUT)
wiringpi.pinMode(BUTTON, GPIO.INPUT)
wiringpi.pullUpDnControl(BUTTON, GPIO.PUD_UP)

while True:
    pressed = wiringpi.digitalRead(BUTTON) == 0    # pulled up, so 0 = pressed
    wiringpi.digitalWrite(LED, GPIO.HIGH if pressed else GPIO.LOW)
    time.sleep(0.02)
```

Making a runtime setting survive reboot

wiringOP changes are lost at power-off. To set pins every time the board starts, run the commands from a small **systemd** service:

```
orangepi@orangepizero2w:~$ sudo nano /etc/systemd/system/gpio-init.service
[Unit]
Description=Set GPIO pin modes at boot
After=multi-user.target

[Service]
Type=oneshot
ExecStart=/bin/sh -c 'gpio mode 2 out; gpio write 2 0; gpio mode 6 in; gpio mode 6 up'

[Install]
WantedBy=multi-user.target
orangepi@orangepizero2w:~$ sudo systemctl daemon-reload
orangepi@orangepizero2w:~$ sudo systemctl enable --now gpio-init.service
```

About alt modes

`gpio mode 2 alt3` really does connect pin 7 to UART4 TX, but no UART driver is running, so no `/dev/ttyS*` appears and nothing transmits. To use a peripheral from software, enable its overlay instead (next sections).

SECTION 10

Method B: enable peripherals with orangepi-config

B

I²C, SPI, UART and PWM on the header are switched off by default. Turning one on means enabling its device-tree overlay and rebooting.

orangepi-config menus (redrawn; the item list on your image is longer)

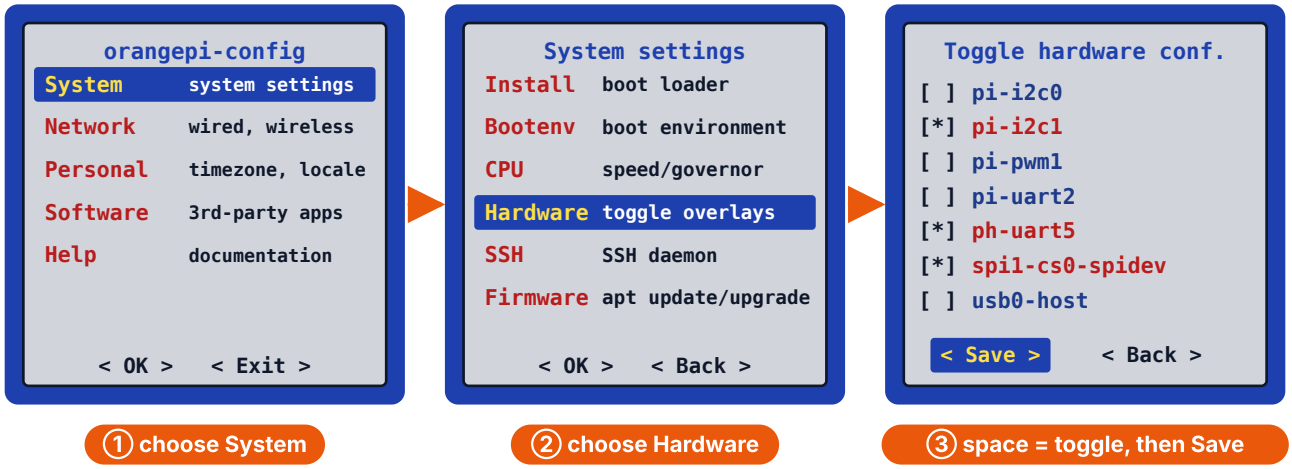


Figure 8. The three menus you pass through (redrawn from the manual's screenshots, §3.19). The Hardware list shows every overlay file installed for your kernel.

1. Run `sudo orangepi-config`.
2. Choose **System**, then **Hardware** ("Toggle hardware configuration").
3. Move with the arrow keys; press `Space` to tick `[*]` or untick an overlay. You can tick several.
4. Press `Tab` to reach **<Save>** and press `Enter`.
5. Choose **<Back>**, then **<Reboot>**. Changes take effect only after the reboot.

orangepi-config name	Turns on	Header pins	Typical device	Conflicts
`pi-i2c0`	I²C0	15 SCL · 22 SDA	`/dev/i2c-0`	not with pi-uart2
`pi-i2c1`	I²C1	5 SCL · 3 SDA	`/dev/i2c-3`	—
`pi-i2c2`	I²C2	28 SCL · 27 SDA	`/dev/i2c-4`	not with pi-uart3
`pi-uart2`	UART2	15 TX · 22 RX	`/dev/ttyS?`	not with pi-i2c0
`pi-uart3`	UART3	28 TX · 27 RX	`/dev/ttyS?`	not with pi-i2c2

orangepi-config name	Turns on	Header pins	Typical device	Conflicts
`pi-uart4`	UART4	7 TX · 16 RX	`/dev/ttyS?`	not with pi-pwm3/4
`ph-uart5`	UART5	11 TX · 13 RX	`/dev/ttyS5`	—
`pi-pwm1`	PWM1	32	pwmchip0/pwm1	—
`pi-pwm2`	PWM2	33	pwmchip0/pwm2	—
`pi-pwm3`	PWM3	7	pwmchip0/pwm3	not with pi-uart4
`pi-pwm4`	PWM4	16	pwmchip0/pwm4	not with pi-uart4
`spi1-cs0-spidev`	SPI1, CS0	19 MOSI · 21 MISO · 23 CLK · 24 CS0	`/dev/spidev1.0`	pick one SPI1 overlay
`spi1-cs1-spidev`	SPI1, CS1	19 · 21 · 23 · 26 CS1	`/dev/spidev1.1`	pick one SPI1 overlay
`spi1-cs0-cs1-spidev`	SPI1, both	19 · 21 · 23 · 24 · 26	`spidev1.0` + `1.1`	pick one SPI1 overlay
`disable-uart0`	frees UART0	8 · 10 become GPIO	console lost	see Section 14

Table 8. Overlays documented for the Orange Pi Zero 2W (manual §3.19 and §5.3). Pin numbers checked against the overlay sources in the Orange Pi Linux 6.1 kernel. "Typical" device names vary; Section 12 shows how to find yours.

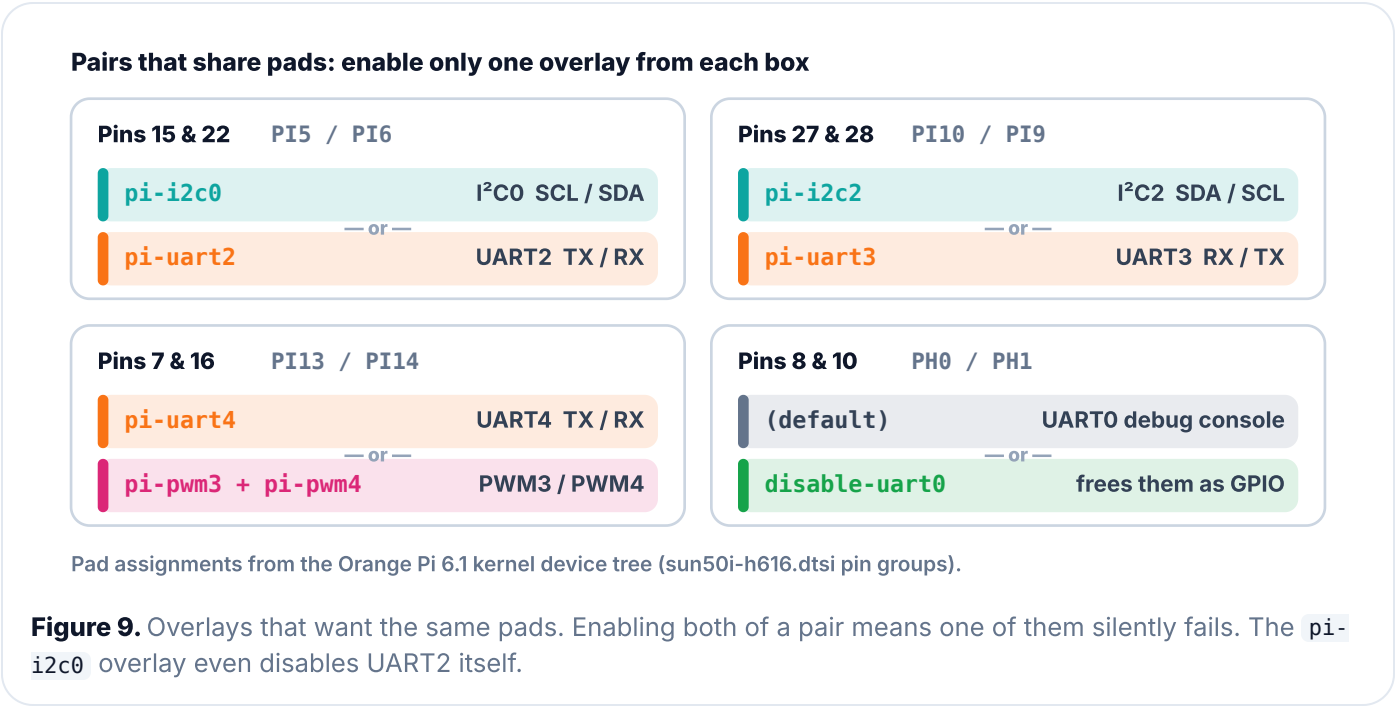


Figure 9. Overlays that want the same pads. Enabling both of a pair means one of them silently fails. The `pi-i2c0` overlay even disables UART2 itself.

Choosing pins for a sensor node
A practical, conflict-free set: **I²C1** (pins 3/5) for environmental sensors, **UART5** (pins 11/13) for a serial particle sensor, **SPI1 CS0** (pins 19/21/23/24) for an SPI device, and the GPIO-only pins for enables and LEDs.

SECTION 11

Method C: edit /boot/orangepiEnv.txt

orangepi-config just edits one line in this file. Editing it yourself is faster once you know the names, and lets you copy a configuration to every node.

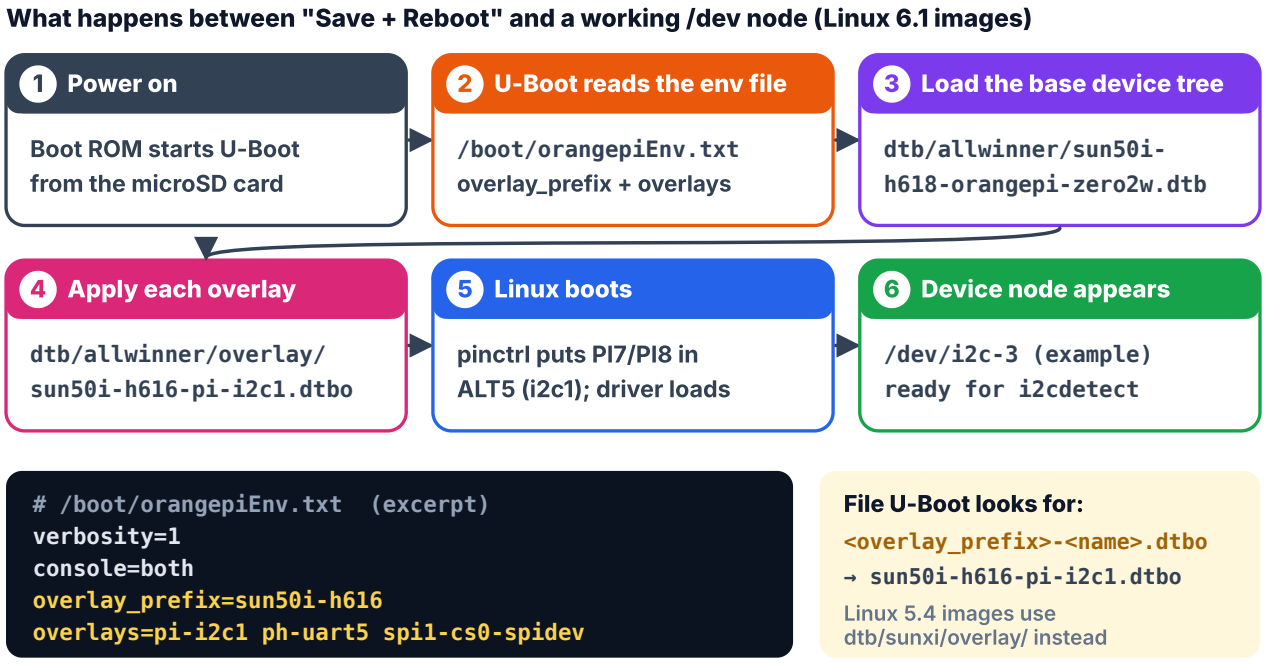


Figure 10. The boot path on Linux 6.1 images, from Orange Pi's U-Boot script (`boot-sun50iw9-next.cmd`). U-Boot builds each file name from `overlay_prefix` and the names on the `overlays=` line.

1 See which overlays exist

```
orangepi@orangepizero2w:~$ ls /boot/dtb/allwinner/overlay/        # Linux 6.1
sun50i-h616-ph-uart5.dtbo  sun50i-h616-pi-i2c1.dtbo  sun50i-h616-spi1-cs0-spidev.dtbo
…
orangepi@orangepizero2w:~$ ls /boot/dtb/sunxi/overlay/            # Linux 5.4 images
```

The name you write is the file name without `sun50i-h616-` and without `.dtbo`: `sun50i-h616-pi-i2c1.dtbo` → `pi-i2c1`.

2 Back up, then edit

```
orangepi@orangepizero2w:~$ sudo cp /boot/orangepiEnv.txt /boot/orangepiEnv.txt.bak
orangepi@orangepizero2w:~$ sudo nano /boot/orangepiEnv.txt
verbosity=1
bootlogo=false
console=both
overlay_prefix=sun50i-h616
overlays=pi-i2c1 ph-uart5 spi1-cs0-spidev      ← add or edit this line
```

Your file may contain more lines (display mode, `rootdev`, `fdtfile`…). Leave them as they are. Save with `Ctrl`+`O`, `Enter`, exit with `Ctrl`+`X`.

3 Reboot and confirm the overlay was applied

```
orangepi@orangepizero2w:~$ sudo reboot
# after logging back in: is the I2C1 controller (i2c@5002400) switched on?
orangepi@orangepizero2w:~$ tr -d '\0' < /proc/device-tree/soc/i2c@5002400/status; echo
okay
# which device owns pads PI7 and PI8 now?
orangepi@orangepizero2w:~$ sudo grep -E "PI7|PI8" /sys/kernel/debug/pinctrl/300b000.pinctrl/pinmux-pins
pin 263 (PI7): device 5002400.i2c function i2c1 group PI7
pin 264 (PI8): device 5002400.i2c function i2c1 group PI8
```

The debugfs folder is named after the pin controller; if the path differs on your image, list it with `sudo ls /sys/kernel/debug/pinctrl/`.

Rule	Why it matters
One `overlays=` line, names separated by spaces	U-Boot loops over the space-separated words.
Keep `overlay_prefix=sun50i-h616`	The Zero 2W uses the H616 overlay files. Changing the prefix to `sun50i-h618` makes every overlay fail to load (reported on the DietPi forum).
A misspelled name is skipped silently	If the file does not exist U-Boot just moves on. Always check with step 3.

Rule	Why it matters
One bad overlay disables all of them	If an overlay fails to apply, U-Boot prints "Error applying DT overlays, restoring original DT" and boots with none.

Table 9. Rules for the overlays line, taken from the U-Boot boot script.

If the board stops booting
Overlay mistakes normally just leave the peripheral disabled. If something else in the file is damaged, restore the backup: log in through the debug UART, or put the card in a Linux PC and copy `orangepiEnv.txt.bak` back over `/boot/orangepiEnv.txt` on the card's root partition (the image has no separate boot partition).

SECTION 12

Prove it works: test each function

Never assume an overlay worked. Each test below takes a minute and needs at most one jumper wire.

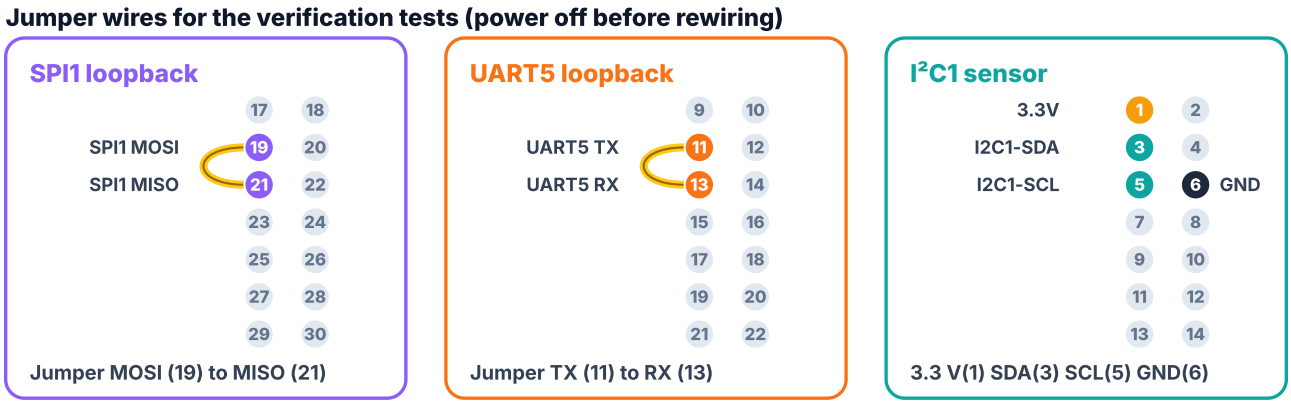


Figure 11. Wiring for the three tests. Loopback tests connect a peripheral's output to its own input, so the board talks to itself and no extra hardware is needed.

I²C (after enabling `pi-i2c1`)

Bus numbers in `/dev` are handed out in the order the controllers start, so I²C1 is usually **not** `/dev/i2c-1`. Map controller addresses to bus numbers first (manual §3.19.4):

```
orangepi@orangepizero2w:~$ ls /sys/devices/platform/soc*/*/i2c-* | grep "i2c-[0-9]"
/sys/devices/platform/soc/5002400.i2c/i2c-3:     ← 5002400 = I2C1 → bus 3
orangepi@orangepizero2w:~$ sudo i2cdetect -y 3
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
70: -- -- -- -- -- -- 76 --                         ← e.g. a BME280 at 0x76
```

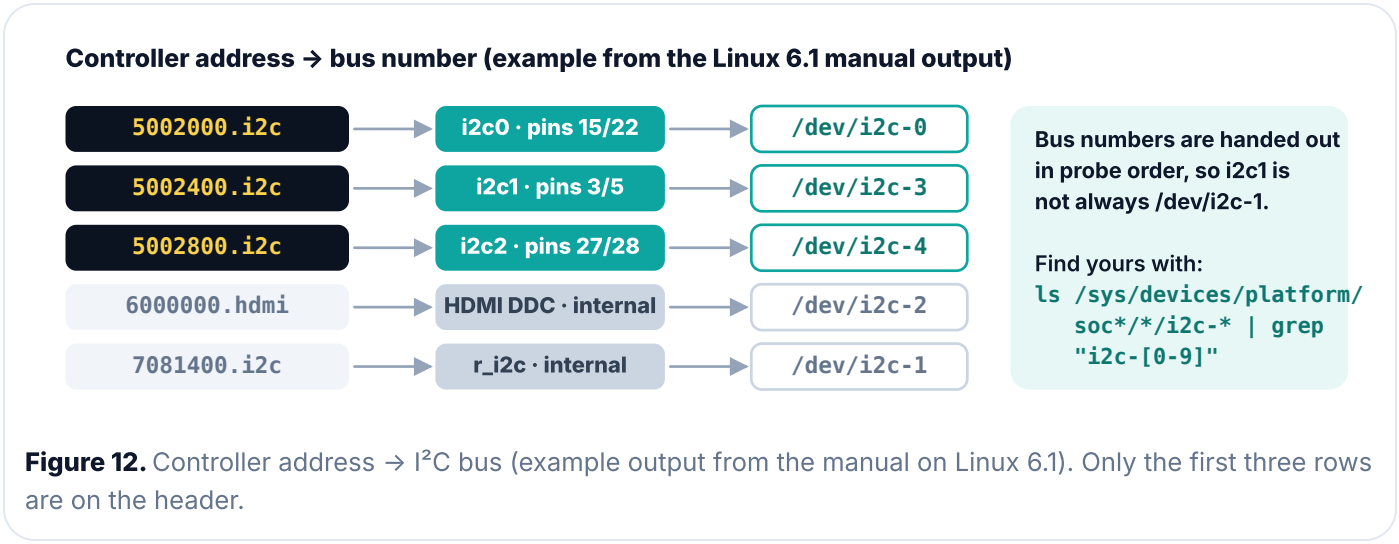


Figure 12. Controller address → I²C bus (example output from the manual on Linux 6.1). Only the first three rows are on the header.

SPI (after enabling `spi1-cs0-spidev`)

```
orangepi@orangepizero2w:~$ ls /dev/spidev1*
/dev/spidev1.0
# jumper pin 19 (MOSI) to pin 21 (MISO), then:
orangepi@orangepizero2w:~$ sudo spidev_test -v -D /dev/spidev1.0
TX | FF FF FF FF FF FF 40 00 00 00 00 95 FF …
RX | FF FF FF FF FF FF 40 00 00 00 00 95 FF …   ← RX equals TX: pass
```

Without the jumper, RX reads all `FF`. Enabling `spi1-cs0-cs1-spidev` instead gives both `/dev/spidev1.0` and `/dev/spidev1.1`.

UART (after enabling `ph-uart5`)

```
orangepi@orangepizero2w:~$ sudo dmesg | grep ttyS            # which name did UART5 get?
5001400.serial: ttyS5 at MMIO 0x5001400 (irq = …) is a 16550A
# jumper pin 11 (TX) to pin 13 (RX), then:
orangepi@orangepizero2w:~$ gpio serial /dev/ttyS5
Out: 0: -> 0
Out: 1: -> 1
Out: 2: -> 2                                    ← Ctrl+C to stop
```

Controller addresses: UART2 `5000800`, UART3 `5000c00`, UART4 `5001000`, UART5 `5001400`. Linux 5.4 images name the ports `/dev/ttyASx` instead of `/dev/ttySx` (manual §3.19.5).

PWM

With wiringOP (needs a recent wiringOP, Section 6). PWM1 is header pin 32, wPi 21:

```
$ gpio mode 21 pwm        # 23 475 Hz, 50 %
$ gpio pwmr 21 2048       # ARR → 25 % duty
$ gpio pwm 21 1024        # CCR → 50 % duty
$ gpio pwmTone 21 20000   # 20 kHz
```

Duty cycle = CCR / ARR (defaults 512 / 1024). Frequencies below about 366 Hz need a larger clock divider (`gpio pwmc`), manual §3.20.

With the kernel driver (enable `pi-pwm1` first). A 50 Hz, 5 % pulse, the classic hobby-servo signal:

```
$ cd /sys/class/pwm/pwmchip0
$ echo 1 | sudo tee export
$ echo 20000000 | sudo tee pwm1/period
$ echo 1000000 | sudo tee pwm1/duty_cycle
$ echo 1 | sudo tee pwm1/enable
```

Times are in nanoseconds: 20 000 000 ns = 20 ms (50 Hz); 1 000 000 ns = 1 ms. Use channel 2, 3 or 4 for PWM2–4 (manual §3.19.6).

SECTION 13

Method D: write your own overlay

D

When the function you need is not in the list, describe it yourself in a few lines of device-tree source, compile it, and add it to the boot configuration.

Anatomy of an overlay

```
/dts-v1/;
/plugin/;

/ {
    fragment@0 {
        target = <&i2c1>;
        __overlay__ {
            pinctrl-names = "default";
            pinctrl-0 = <&i2c1_pi_pins>;
            status = "okay";
        };
    };
};
```

This is the complete source of Orange Pi's own `pi-i2c1` overlay. Read it as: *"take the node labelled `i2c1` in the base tree, attach it to the pin group `i2c1_pi_pins` (pads PI7 and PI8), and set its status to okay"*.

- **target** names an existing node by its label.
- **pinctrl-0** chooses the pads, and therefore the ALT function.
- **status = "okay"** starts the driver; `"disabled"` stops it.
- Pin groups such as `uart5_ph_pins` are defined in the kernel file `sun50i-h616.dtsi` .

Example 1: a DS18B20 1-Wire temperature probe on pin 12

1-Wire is a one-data-wire bus used by the popular DS18B20 waterproof temperature probe. Orange Pi provides no 1-Wire overlay, but the kernel's `w1-gpio` driver (built as a module in the Orange Pi 6.1 kernel) can run it on any GPIO. In `<&pio 8 1 0>`, 8 is bank I, 1 is the pin index (so PI1, header pin 12) and 0 means active-high. Wire the probe: **VDD → pin 17** (3.3 V), **GND → pin 14**, **DQ → pin 12** (PI1), with a **4.7 kΩ resistor between pin 12 and pin 17**.

```
orangepi@orangepizero2w:~$ nano w1-pi1.dts
/dts-v1/;
/plugin/;

/ {
    fragment@0 {
        target-path = "/";
        __overlay__ {
            onewire-pi1 {
                compatible = "w1-gpio";
                gpios = <&pio 8 1 0>;  // PI1 = pin 12
                status = "okay";
            };
        };
    };
};
orangepi@orangepizero2w:~$ sudo orangepi-add-overlay w1-pi1.dts
Compiling the overlay
Copying the compiled overlay file to /boot/overlay-user/
Reboot is required to apply the changes
orangepi@orangepizero2w:~$ sudo reboot
orangepi@orangepizero2w:~$ ls /sys/bus/w1/devices/
28-0000xxxxxxxx  w1_bus_master1
orangepi@orangepizero2w:~$ cat /sys/bus/w1/devices/28-*/w1_slave
… : crc=… YES
… t=23125                                  ← 23.125 °C
```

What orangepi-add-overlay did
It compiled the source with `dtc -@`, copied the result to `/boot/overlay-user/w1-pi1.dtbo`, and appended `user_overlays=w1-pi1` to `/boot/orangepiEnv.txt`. User overlays load after the kernel-provided ones. You can do the same by hand: `dtc -@ -I dts -O dtb -o w1-pi1.dtbo w1-pi1.dts`, then copy the file and edit the line yourself.

Example 2: set the I²C1 bus speed

The kernel runs I²C buses at 100 kHz unless told otherwise. A user overlay can change that, for example to 400 kHz for a bus that only has fast sensors, or down to 50 kHz for a long cable. Keep `pi-i2c1` enabled in `overlays=`; this overlay only adds one property.

```
/dts-v1/;
/plugin/;

/ {
    fragment@0 {
        target = <&i2c1>;
        __overlay__ {
            clock-frequency = <400000>;   // Hz
        };
    };
};
```

Check every device on the bus first
A bus runs at one speed for all devices. Some sensors, such as the Sensirion SCD30 and SPS30, are limited to 100 kHz on I²C. Speeding the bus up for one sensor can break another.

If orangepi-add-overlay says the headers are broken
The script prefers the `dtc` inside the kernel headers. If it reports "kernel headers are not installed properly", either install the headers that ship with the image (`sudo dpkg -i /opt/linux-headers*.deb` , manual §3.32) or use the manual `dtc` route above with the `device-tree-compiler` package.

SECTION 14

Giving pins back

Turning a peripheral off returns its pins to plain GPIO. This is how you resolve conflicts and free pins for new uses.

1. **Kernel overlays:** untick the item in `orangepi-config` , or delete its name from the `overlays=` line. Reboot.
2. **Your own overlays:** delete the name from the `user_overlays=` line (and, if you like, the file in `/boot/overlay-user/`). Reboot.
3. **Check:** `gpio readall` shows the pins as `OFF` again, and the `/dev` node is gone.

Freeing pins 8 and 10 (the debug UART)

The `disable-uart0` overlay switches off UART0 and returns pads PH0 and PH1 (pins 8 and 10) to GPIO. The manual warns against using UART0 as an ordinary serial port while it is the debug console.

You lose your rescue console
With UART0 disabled you can no longer log in over the serial adapter when networking fails. Only do this on a board where SSH is working, and keep a second, unmodified microSD card for emergencies.

Overlays that quietly disable UART0
`ph-i2c1` and `ph-pwm34` (shipped for the whole H616/H618 family but not listed in the Zero 2W manual) also contain a fragment that disables UART0. Appendix B marks such overlays "undocumented".

Back to a clean slate

To undo every overlay at once, for example before handing a board to the next student, remove both overlay lines and reboot. A fresh image has neither line; only `overlay_prefix=` is needed.

```
orangepi@orangepizero2w:~$ sudo cp /boot/orangepiEnv.txt /boot/orangepiEnv.txt.bak
orangepi@orangepizero2w:~$ sudo sed -i -e '/^overlays=/d' -e '/^user_overlays=/d'
/boot/orangepiEnv.txt
orangepi@orangepizero2w:~$ grep overlay /boot/orangepiEnv.txt
overlay_prefix=sun50i-h616          # the only overlay line left
orangepi@orangepizero2w:~$ sudo reboot
```

Label the board
Stick a label on each lab board listing its enabled overlays (the output of `grep overlays /boot/orangepiEnv.txt`). The next person then knows which pins are taken before wiring anything.

PART 4

Reference

04

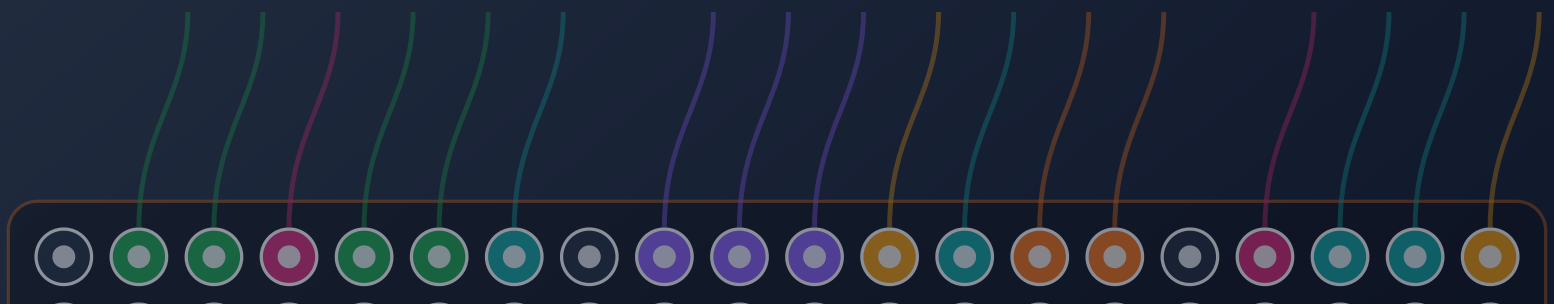
Troubleshooting, a one-page cheat sheet, the full pin-function table generated from the Linux pin-control driver, and the catalogue of overlay files on a Linux 6.1 image.

15 · Troubleshooting
Symptom → cause → fix

16 · Cheat sheet
Every command from this guide on one page

A · Pin-function table
All ALT functions for the 28 header pads

B · Overlay catalogue
Every .dtbo you will see in orangepi-config



SECTION 15

Troubleshooting

Work from the hardware upward: power and wiring first, then numbering, then the device tree, then your code.

Symptom	Likely cause	Fix
`gpio: command not found`	wiringOP missing (custom or third-party image)	Build it from source (Section 6, step 3).
`gpio readall` shows the wrong board or no table	Old wiringOP that does not know the Zero 2W	Rebuild from the `next` branch.
Pin does not change when written	Wrong numbering system (physical number used as wPi)	Check the pin table; use `gpio -1` for physical numbers.
"Permission denied" or no effect from Python	wiringOP needs root	Run with `sudo`.
Input reads random 0s and 1s	Floating input (no pull resistor)	`gpio mode N up` or `down`, or add an external resistor.
No `/dev/i2c-*`, `/dev/spidev*` or new `ttyS` after reboot	Overlay not applied: typo, wrong prefix, or not saved	Check the name against `ls /boot/dtb/allwinner/overlay/`; check `/proc/device-tree/…/status` (Section 11).
All overlays stopped working at once	One overlay failed to apply, so U-Boot restored the original tree	Remove the last one you added; watch the serial console during boot for the error.
`i2cdetect` shows nothing	Wrong bus number; SDA/SCL swapped; sensor unpowered; no pull-ups on the module	Map the bus (Section 12); check wiring and 3.3 V at the sensor.
Two peripherals, only one works	Pad conflict (e.g. `pi-i2c0` + `pi-uart2`)	Enable only one of each pair (Section 10).
Serial console garbled or dead after a GPIO experiment	wiringOP changed pins 8/10	Reboot; never write to wPi 3 and 4.
`orangepi-add-overlay` fails	Kernel headers or `dtc` missing	Install `device-tree-compiler`, or the headers from `/opt`.
Board resets when a sensor or motor starts	Weak 5 V supply or too much current drawn from header pins	Use a 5 V / 3 A supply; power motors and fans separately.

Table 10. Common problems and fixes.

SECTION 16

Cheat sheet

Print this page and keep it next to the bench.

Look

```
gpio readall                          # state of every pin
gpio -v                               # wiringOP version
cat /boot/orangepiEnv.txt             # which overlays are enabled
ls /boot/dtb/allwinner/overlay/       # which overlays exist (Linux 6.1)
gpiodetect ; gpioinfo                 # GPIO lines claimed by kernel drivers
ls /dev/i2c-* /dev/spidev* /dev/ttyS* # device nodes
```

GPIO at runtime (wPi numbers)

```
gpio mode 2 out ; gpio write 2 1
gpio toggle 2
gpio mode 6 in ; gpio mode 6 up
gpio read 6
gpio -1 mode 7 out     # physical
gpio -g mode 269 out   # Linux GPIO
```

PWM with wiringOP (pin 32 = wPi 21)

```
gpio mode 21 pwm
gpio pwmr 21 2048      # ARR
gpio pwm 21 1024       # CCR
gpio pwmc 21 5         # divider
gpio pwmTone 21 20000  # Hz
```

Change pin functions (persistent)

```
sudo orangepi-config                # System → Hardware → Space → Save → Back → Reboot
sudo nano /boot/orangepiEnv.txt     # or edit by hand:
    overlay_prefix=sun50i-h616
    overlays=pi-i2c1 ph-uart5 spi1-cs0-spidev
sudo orangepi-add-overlay my.dts    # your own overlay → user_overlays=my
sudo reboot
```

Test

```
ls /sys/devices/platform/soc*/*/i2c-* | grep "i2c-[0-9]"   # map I2C buses
sudo i2cdetect -y 3                                        # scan bus 3
sudo spidev_test -v -D /dev/spidev1.0                      # jumper 19↔21
sudo dmesg | grep ttyS                                     # find UART names
gpio serial /dev/ttyS5                                     # jumper 11↔13
```

Safe pin choices

GPIO only: 12, 18, 29, 31, 35, 36, 37, 38, 40 · I²C1: 3, 5 · UART5: 11, 13 · SPI1: 19, 21, 23, 24 (+26) · PWM1/PWM2: 32, 33 · avoid 8 and 10 (debug console).

APPENDIX A

Full pin-function table

Every function the H618 can route to each of the 28 header pads, read from the Linux 6.1 pin-control driver. Bold = supported by an Orange Pi overlay on this board.

Pin	Pad	GPIO	wPi	ALT2	ALT3	ALT4	ALT5
3	PI8	264	0	emac0 ETXD2	uart2 CTS	ts0 DVLD	**i2c1 SDA**
5	PI7	263	1	emac0 ETXD3	uart2 RTS	ts0 SYNC	**i2c1 SCK**
7	PI13	269	2	emac0 ECLKIN	**uart4 TX**	ts0 D4	**pwm3**
8	PH0	224	3	**uart0 TX**	—	pwm3	i2c1 SCK
10	PH1	225	4	**uart0 RX**	—	pwm4	i2c1 SDA
11	PH2	226	5	**uart5 TX**	spdif MCLK	pwm2	i2c2 SCK
12	PI1	257	6	emac0 ERXD2	dmic DATA0	i2s0 BCLK	hdmi HSDA
13	PH3	227	7	**uart5 RX**	—	pwm1	i2c2 SDA
15	PI5	261	8	emac0 ERXCTL	**uart2 TX**	ts0 CLK	**i2c0 SCK**
16	PI14	270	9	emac0 MDC	**uart4 RX**	ts0 D5	**pwm4**
18	PH4	228	10	—	spdif OUT	—	i2c3 SCK
19	PH7	231	11	uart2 RTS	i2s3 SYNC	**spi1 MOSI**	i2c4 SDA
21	PH8	232	12	uart2 CTS	i2s3_dout0 DO0	**spi1 MISO**	i2s3_din1 DI1
22	PI6	262	13	emac0 ENULL	**uart2 RX**	ts0 ERR	**i2c0 SDA**
23	PH6	230	14	uart2 RX	i2s3 BCLK	**spi1 CLK**	i2c4 SCK
24	PH5	229	15	uart2 TX	i2s3 MCLK	**spi1 CS0**	i2c3 SDA
26	PH9	233	16	—	i2s3_din0 DI0	**spi1 CS1**	i2s3_dout1 DO1
27	PI10	266	17	emac0 ETXD0	**uart3 RX**	ts0 D1	**i2c2 SDA**
28	PI9	265	18	emac0 ETXD1	**uart3 TX**	ts0 D0	**i2c2 SCK**
29	PI0	256	19	emac0 ERXD3	dmic CLK	i2s0 MCLK	hdmi HSCL
31	PI15	271	20	emac0 MDIO	uart4 RTS	ts0 D6	clock CLK_FANOUT0
32	PI11	267	21	emac0 ETXCK	uart3 RTS	ts0 D2	**pwm1**
33	PI12	268	22	emac0 ETXCTL	uart3 CTS	ts0 D3	**pwm2**
35	PI2	258	23	emac0 ERXD1	dmic DATA1	i2s0 SYNC	hdmi HCEC
36	PC12	76	24	nand0 DQS	—	—	—
37	PI16	272	25	emac0 EPHY_CLK	uart4 CTS	ts0 D7	clock CLK_FANOUT1
38	PI4	260	26	emac0 ERXCK	dmic DATA3	i2s0_din0 DI0	i2s0_dout1 DO1
40	PI3	259	27	emac0 ERXD0	dmic DATA2	i2s0_dout0 DO0	i2s0_din1 DI1

Table 11. ALT functions per pad (`drivers/pinctrl/sunxi/pinctrl-sun50i-h616.c`, Orange Pi 6.1 kernel). ALT0 = input, ALT1 = output, ALT6 = external interrupt (`Px_EINTn`) on every pad, ALT7 = disabled. emac0 = Ethernet MAC, ts0 = transport-stream input, dmic = digital microphone, i2s = audio serial bus; these need extra device-tree work and are not covered here.

Using this table
To find out whether a function can reach the header, look for it in the ALT columns. For example, UART2 appears on PI5/PI6 (pins 15/22, ALT3) and also on PH5/PH6 (pins 24/23, ALT2), which is why two different UART2 overlays exist.

APPENDIX B

Overlay catalogue (Linux 6.1)

.dtbo

orangepi-config lists every `sun50i-h616-*.dtbo` file on the card, including ones built for the whole H616/H618 family. This table shows which ones the Zero 2W manual documents.

Overlay name(s)	What it does	Header pins	Status
pi-i2c0 / pi-i2c1 / pi-i2c2	I²C0/1/2 on PI5-6, PI7-8, PI9-10	15/22 · 5/3 · 28/27	documented
pi-uart2 / pi-uart3 / pi-uart4	UART2/3/4 on PI5-6, PI9-10, PI13-14	15/22 · 28/27 · 7/16	documented
ph-uart5	UART5 on PH2-3	11/13	documented
pi-pwm1 … pi-pwm4	PWM1-4 on PI11, PI12, PI13, PI14	32 · 33 · 7 · 16	documented
spi1-cs0-spidev · spi1-cs1-spidev · spi1-cs0-cs1-spidev	SPI1 on PH6-8 + CS0 PH5 / CS1 PH9	19 · 21 · 23 · 24 / 26	documented
disable-uart0	turns off the debug UART	frees 8 · 10	documented
zero2w-disable-led	turns off the green status LED	—	documented
usb0-host	puts USB-C port 0 into host mode	—	documented
ph-uart2	UART2 on PH5-6	24/23 (clashes with SPI1)	undocumented
ph-i2c1	I²C1 on PH0-1, and disables UART0	8/10 (debug UART)	undocumented
ph-i2c2	I²C2 on PH2-3	11/13 (clashes with UART5)	undocumented

Overlay name(s)	What it does	Header pins	Status
`ph-i2c3`	I²C3 on PH4-5	18/24 (I²C3 is used on-board)	undocumented
`ph-i2c4`	I²C4 on PH6-7	23/19 (clashes with SPI1)	undocumented
`ph-pwm12 · ph-pwm34`	enable the PWM block; ph-pwm34 also disables UART0	—	undocumented
`spi0-spidev`	SPI0 spidev; disables the on-board SPI flash	on-board flash bus	undocumented
`disable-leds`	generic LED overlay (the Zero 2W uses zero2w-disable-led)	—	undocumented
`ir`	IR receiver on PH10 (24-pin expansion board)	—	not header
`gpu`	enables the GPU node	—	not header

Table 12. All overlays built by the Orange Pi Linux 6.1 kernel (`arch/arm64/boot/dts/allwinner/overlay/Makefile`), with the pads each one claims. "Documented" = listed for the Zero 2W in user manual v1.3; "undocumented" overlays are built for the whole chip family and may clash with how this board uses its pins.

SOURCES

Where every fact comes from

Pin numbers, overlay names and file paths were cross-checked against at least two of these.

Vendor documentation

[1] Shenzhen Xunlong Software Co. *Orange Pi Zero 2W User Manual*, v1.3 (H618). §1.4–1.6 hardware; §2.8 debug serial; §3.1 images; §3.4 login; §3.17 40-pin table; §3.18 wiringOP; §3.19–3.21 GPIO/SPI/I²C/UART/PWM tests; §3.32 kernel headers; §5.3 DT overlays.

[2] Orange Pi. *Orange Pi Zero 2W product page*, orangepi.org (specifications and downloads).

[3] MINTS lab shared Drive folder: *OPi_ZERO 2W_SCH.pdf*, *ZERO_2W_24PIN_EXTEND_SCH.pdf*, *Opi ZERO 2W DXF.rar* (schematics and outline).

Source code (github.com/orangepi-xunlong)

[4] *linux-orangepi*, branch orange-pi-6.1-sun50iw9: `sun50i-h616.dtsi` (pin groups), `sun50i-h618-orangepi-zero2w.dts` (board tree), `overlay/*.dts` and `overlay/Makefile`, `drivers/pinctrl/sunxi/pinctrl-sun50i-h616.c` (ALT functions).

[5] *orangepi-build*, branch next: `boot-sun50iw9-next.cmd` (U-Boot overlay loading), `sun50iw9.conf` (overlay prefix), `orangepi-add-overlay`, orangepi-config `debian-config-jobs` (Hardware menu).

[6] *wiringOP*, branch next: `wiringPi.c` (Zero 2W pin maps), `gpio.c` (command modes). *wiringOP-Python*, branch next.

Community reports

[7] DietPi forum, "[Orange Pi Zero 2W] How to enable UART5?" (2025): `overlay_prefix` must stay `sun50i-h616`; UART5 appears as `ttyS5`.

[8] Ubuntu and Debian package archives: `gpiod` 1.6.3 in Debian 12 and Ubuntu 24.04.

When this guide and your board disagree

Trust, in order: the schematic, `gpio readall` on your board, then this guide. Image updates can rename overlays or device nodes; `ls /boot/dtb/allwinner/overlay/` and `dmesg` always show the truth for the image you are running.